EJERCICIO 6 - Ciclos, camino crítico y clasificación topológica

Estructura:
- El grafo es dirigido. Cada vértice tiene su lista de aristas salientes.
- Cada arista tiene origen, destino y un peso. En el camino crítico, el peso
  es la duración de la tarea: las tareas son los arcos y los vértices son hitos
  del proyecto.
- Se usan: sucesores(v), predecesores(v), gradoDeEntrada(v) y gradoDeSalida(v).

Precondición común a todas las operaciones: el grafo está creado (puede estar
vacío, salvo que se diga otra cosa).

En los órdenes, V es la cantidad de vértices y E la cantidad de aristas.

---
1. SABER SI EL GRAFO TIENE CICLOS

Lenguaje natural:
Se hace una búsqueda en profundidad desde cada vértice que todavía no se
visitó. Se guardan dos conjuntos: los visitados, y los que están en el camino
(los pendientes de la recursión actual). Al mirar un arco, si el destino está
en el camino, se volvió a un antecesor: es un arco de retroceso y hay un ciclo.
Si el destino ya se visitó pero no está en el camino, no hay ciclo (se llegó
a él por otro lado). Si no se visitó, se sigue desde él.

Precondiciones: la común.

Postcondiciones:
- El grafo no cambia.
- Devuelve VERDADERO si y solo si hay al menos un ciclo.
- Un grafo vacío no tiene ciclos. Un lazo (A -> A) es un ciclo.

Pseudocódigo:

```
tieneCiclos(grafo) devuelve booleano
Comienzo
    visitados <- conjunto vacío
    enElCamino <- conjunto vacío
    PARA CADA vertice EN vertices(grafo) HACER
        SI vertice NO ESTA EN visitados ENTONCES
            SI bpfTieneCiclo(vertice) ENTONCES
                devolver VERDADERO
            FIN SI
        FIN SI
    FIN PARA
    devolver FALSO
Fin

bpfTieneCiclo(actual) devuelve booleano
Comienzo
    visitados.agregar(actual)
    enElCamino.agregar(actual)
    PARA CADA sucesor EN sucesores(actual) HACER
        SI sucesor ESTA EN enElCamino ENTONCES
            devolver VERDADERO           // arco de retroceso
        FIN SI
        SI sucesor NO ESTA EN visitados ENTONCES
            SI bpfTieneCiclo(sucesor) ENTONCES
```

```
                devolver VERDADERO
            FIN SI
        FIN SI
    FIN PARA
    enElCamino.quitar(actual)
    devolver FALSO
Fin
```

Orden del tiempo de ejecución:
Cada vértice se visita una sola vez y cada arista se mira una sola vez.
Total: O(V + E).

Resultado con aeropuertos_2.txt y conexiones_2.txt: el grafo SÍ tiene ciclos
(por ejemplo San_Pablo -> Florianopolis -> San_Pablo).

---

2. CAMINO CRÍTICO

Lenguaje natural:
Una secuencia es un camino desde un vértice inicial (sin predecesores) hasta
uno final (sin sucesores). El camino crítico es la secuencia más larga.
Primero se obtiene la clasificación topológica (operación 3), que además
comprueba que no haya ciclos. Se recorren los vértices en ese orden: cuando
se llega a un vértice, ya se conoce el costo final de todos sus predecesores,
así que su costo más largo es el máximo de (costo del predecesor + peso del
arco). Se guarda qué predecesor dio ese máximo. El vértice final de mayor costo
cierra el camino crítico, y yendo hacia atrás por esos predecesores se arma.
Después se listan todas las secuencias con una búsqueda en profundidad desde
cada vértice inicial, y a cada una se le calcula la holgura:
holgura = costo del camino crítico - costo de la secuencia.

Precondiciones: el grafo no es vacío y es acíclico. Los pesos son >= 0.

Postcondiciones:
- El grafo no cambia.
- Devuelve el costo del camino crítico y los vértices que lo forman.
- Devuelve todas las secuencias con su costo y su holgura.
- Las secuencias de costo máximo tienen holgura 0.

Pseudocódigo:

```
caminoCritico(grafo) devuelve (costo, camino, secuencias)
Comienzo
    orden <- clasificacionTopologica(grafo)
    PARA CADA vertice EN orden HACER
        costoMasLargo[vertice] <- 0
        predecesorCritico[vertice] <- nulo
    FIN PARA

    PARA CADA vertice EN orden HACER
        PARA CADA arco EN aristasSalientes(vertice) HACER
            nuevoCosto <- costoMasLargo[vertice] + peso(arco)
            SI nuevoCosto > costoMasLargo[destino(arco)] ENTONCES
                costoMasLargo[destino(arco)] <- nuevoCosto
                predecesorCritico[destino(arco)] <- vertice
            FIN SI
        FIN PARA
    FIN PARA
```

```
    costoCritico <- -1
    verticeFinal <- nulo
    PARA CADA vertice EN vertices(grafo) HACER
        SI gradoDeSalida(vertice) = 0 AND costoMasLargo[vertice] > costoCritico ENTONCES
            costoCritico <- costoMasLargo[vertice]
            verticeFinal <- vertice
        FIN SI
    FIN PARA

    camino <- lista vacía
    actual <- verticeFinal
    MIENTRAS actual <> nulo HACER
        camino.agregarAlPrincipio(actual)
        actual <- predecesorCritico[actual]
    FIN MIENTRAS

    secuencias <- lista vacía
    PARA CADA inicio EN vertices(grafo) HACER
        SI gradoDeEntrada(inicio) = 0 ENTONCES
            recorrer(inicio, [inicio], 0)
        FIN SI
    FIN PARA

    devolver (costoCritico, camino, secuencias)
Fin

recorrer(actual, caminoActual, costoActual)
Comienzo
    SI gradoDeSalida(actual) = 0 ENTONCES
        secuencias.agregar(copia de caminoActual, costoActual, costoCritico - costoActual)
    SINO
        PARA CADA arco EN aristasSalientes(actual) HACER
            caminoActual.agregarAlFinal(destino(arco))
            recorrer(destino(arco), caminoActual, costoActual + peso(arco))
            caminoActual.quitarUltimo()      // se deshace el paso para probar otro arco
        FIN PARA
    FIN SI
Fin
```

Orden del tiempo de ejecución:
Calcular el camino crítico es la clasificación topológica, $O(V \cdot (V + E))$ (ver operación 3), más una pasada por cada vértice y cada arista. Esa pasada sería $O(V + E)$, pero obtener las aristas salientes de un vértice cuesta $O(V)$ porque hay que buscarlo, así que queda $O(V^2 + E)$. Total: $O(V \cdot (V + E))$. Listar las secuencias puede ser exponencial, porque la cantidad de caminos puede crecer muy rápido; no se puede evitar si hay que listarlas todas.

Ejemplo: A -3-> B, A -2-> C, B -4-> D, C -6-> D, D -1-> E.
- A, C, D, E cuesta 9 (crítico, holgura 0).
- A, B, D, E cuesta 8 (holgura 1).

---

3. CLASIFICACIÓN TOPOLÓGICA

Lenguaje natural:
Un vértice tiene que aparecer después de todos los que lo preceden. Se hace una

búsqueda en profundidad siguiendo los predecesores (es lo mismo que invertir los arcos) y cada vértice se agrega a la lista al salir de la recursión, es decir, cuando todo lo que lo precede ya está en la lista.

Precondiciones: el grafo es acíclico.

Postcondiciones:
- El grafo no cambia.
- Devuelve una lista con todos los vértices, una sola vez cada uno.
- Para todo arco u -> v, u aparece antes que v en la lista.

Pseudocódigo:

```
clasificacionTopologica(grafo) devuelve Lista
Comienzo
    SI tieneCiclos(grafo) ENTONCES
        error "el grafo tiene ciclos"
    FIN SI
    resultado <- lista vacía
    visitados <- conjunto vacío
    PARA CADA vertice EN vertices(grafo) HACER
        SI vertice NO ESTA EN visitados ENTONCES
            bpfTopologico(vertice)
        FIN SI
    FIN PARA
    devolver resultado
Fin

bpfTopologico(actual)
Comienzo
    visitados.agregar(actual)
    PARA CADA predecesor EN predecesores(actual) HACER
        SI predecesor NO ESTA EN visitados ENTONCES
            bpfTopologico(predecesor)
        FIN SI
    FIN PARA
    resultado.agregarAlFinal(actual)         // salida de la recursión
Fin
```

Orden del tiempo de ejecución:
Una búsqueda en profundidad visita cada vértice y cada arista una sola vez, y tieneCiclos es O(V + E). Pero esta búsqueda sigue los predecesores, y como cada vértice solo guarda sus aristas salientes, obtener los predecesores de un vértice obliga a recorrer todas las aristas del grafo: O(V + E) por vértice. Total: O(V · (V + E)). Si además se guardaran las aristas entrantes de cada vértice, sería O(V + E).